

北京理工大学

本科生毕业设计（论文）

面向桌面操作的 VLA 机械臂抓取与控制研究

Study on Grasping Strategy and Motion Control of VLA

Robotic Arm Oriented to Desktop Operation

学 院： 自动化学院

专 业： 自动化

班 级： 06212202

学生姓名： 唐元昊

学 号： 1120221074

指导教师： 方浩

2026 年 5 月 6 日

原创性声明

特此申明。

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

关于使用授权的声明

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

指导老师签名: _____ 日期: _____ 年 _____ 月 _____ 日

面向桌面操作的 VLA 机械臂抓取与控制研究

摘 要

本文……。

摘要正文选用模板中的样式所定义的“正文”，每段落首行缩进 2 个字符；或者手动设置成每段落首行缩进 2 个汉字，字体：宋体，字号：小四，行距：固定值 22 磅，间距：段前、段后均为 0 行。阅后删除此段。

摘要是一篇具有独立性和完整性的短文，应概括而扼要地反映出本论文的主要内容。包括研究目的、研究方法、研究结果和结论等，特别要突出研究结果和结论。中文摘要力求语言精炼准确，本科生毕业设计（论文）摘要建议 300-500 字。摘要中不可出现参考文献、图、表、化学结构式、非公知公用的符号和术语。英文摘要与中文摘要的内容应一致。阅后删除此段。

关键词：北京理工大学；本科生；毕业设计（论文）——请在“main.tex”开头设置

Study on Grasping Strategy and Motion Control of VLA Robotic Arm Oriented to Desktop Operation

Abstract

In order to study……

Abstract 正文设置成每段落首行缩进 2 字符，字体：Times New Roman，字号：小四，行距：固定值 22 磅，间距：段前、段后均为 0 行。阅后删除此段。

Key Words: BIT; Undergraduate; Graduation Project (Thesis)

目 录

摘 要	I
Abstract	II
第 1 章 一级题目	1
1.1 二级题目	1
1.1.1 三级题目	1
1.2 字体效果表格	2
1.3 常见问题和疑难解答	2
第 2 章 Franka 机械臂系统集成与 LeRobot 接口扩展	3
2.1 系统总体架构	3
2.2 Franka 实时控制与底层通信	4
2.2.1 控制环路与实时性	4
2.2.2 套接字协议与上下位机分工	5
2.3 LeRobot 通用机器人接口扩展	5
2.3.1 框架约定与工厂注册	5
2.3.2 FrankaFER 类设计要点	5
2.3.3 带末端夹爪的复合机器人	6
2.4 三维鼠标遥操作方案	6
2.4.1 硬件与数据获取	6
2.4.2 累积位姿与死区处理	6
2.4.3 逆运动学与 Teleoperator 接口	7
2.5 同构主从臂遥操作、配对标定与数据采集	7
2.5.1 同构构型的工程动机	7
2.5.2 关节空间映射模型	8
2.5.3 两阶段配对标定与自动修正	8
2.5.4 与录制管线及数据质量的衔接	9
2.6 开源与可复现性说明	9
第 3 章 桌面抓取示教数据采集与 LeRobot 数据集规范	11
3.1 采集任务与录制流程概要	11
3.2 LeRobot v2.1 目录结构与元数据	12
3.3 帧级特征模式与模仿学习接口	13
3.4 数据规模、质量与后续训练衔接	14

3.5 基于 Rerun 的可视化核验	14
3.6 本章小结	14
第 4 章 基于 $\pi_{0.5}$ 的桌面抓取策略训练与实验分析	16
4.1 $\pi_{0.5}$ 的核心原理框架	16
4.1.1 从 VLA 到分层推理	16
4.1.2 统一多模态序列建模	17
4.1.3 主干网络与动作专家	18
4.1.4 动作专家中的 Flow Matching 机制	19
4.1.5 离散动作预训练与连续动作后训练	20
4.1.6 异构数据协同训练的意义	21
4.2 面向 Franka 桌面抓取的模型适配与训练实现	21
4.2.1 任务观测与动作空间重定义	21
4.2.2 训练配置与关键超参数	22
4.2.3 训练产物与 checkpoint 组织	23
4.2.4 本文对 $\pi_{0.5}$ 的针对性修改	23
4.3 在线推理执行策略与实机效果分析	24
4.3.1 两种在线执行脚本的差异	24
4.3.2 成功率差异的原因分析	25
4.3.3 实验现象与任务完成质量	26
4.4 模型变体设计、效果预估与行为观察	26
4.4.1 全量微调基线	26
4.4.2 冻结部分主干的轻量化变体	26
4.4.3 short_memory 版本的行为特征	27
4.4.4 模型变体的综合比较	28
4.4.5 对后续工作的启示	28
4.5 本章小结	29
结 论	30
参考文献	31
附 录	32
附录 A L ^A T _E X 环境的安装	32
附录 B BITHesis 使用说明	32
致 谢	33

第 1 章 一级题目

1.1 二级题目

1.1.1 三级题目

正文……[1]……[2]

正文部分：宋体、小四；正文行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

图、表居中，图注标在图下方，表头标在表上方，宋体、五号、居中，1.25 倍行距，间距段前段后均为 0 行，图表与上下文之间各空一行。阅后删除此段。

图-示例：（阅后删除此段）



图 1-1 标题序号

表-示例：（阅后删除此段）

表 1-1 统计表

项目	产量	销量	产值	比重
手机	1000	10000	500	50%
计算机	5500	5000	220	22%
笔记本电脑	1100	1000	280	28%
合计	17600	16000	1000	100%

公式标注应于该公式所在行的最右侧。对于较长的公式只可在符号处（+、-、*、/、 $\leq$ 、 $\geq$ 等）转行。在文中引用公式时，在标号前加“式”，如式（1-2）。阅后删除此段。

公式-示例：（阅后删除此段）

$$LRI = 1/\sqrt{1 + \left(\frac{\mu_R}{\mu_s}\right)^2 \left(\frac{\delta_R}{\delta_s}\right)^2} \quad (1-1)$$

1.2 字体效果表格

表 1-2 字体效果表格

	Regular	Bold	Italic	Bold Italic
宋体	宋体	宋体粗体	楷体	楷书粗斜体
黑体	黑体	黑体粗体	黑体斜体	黑体粗斜体
楷体	楷书	楷书粗体	斜体楷体	楷书粗斜体
Serif(Roman/Normal)	Regular	Bold	<i>Italic</i>	<i>Bold Italic</i>
Sans Serif	Regular	Bold	<i>Bold</i>	<i>Bold</i>
Typewriter	Regular	Bold	<i>Bold</i>	<i>Bold</i>
Math	<i>Regular</i> Roman	Bold	<i>Italic</i>	<i>BoldItalic</i>

- **宋体粗体** 在 Windows 下会成为黑体。这是因为 Windows 的中易宋体没有粗体字重而进行的妥协。如果想要获得宋体粗体的样式，请在配置中开启伪粗体选项。
- **粗斜体** 的效果是因操作系统字体而定的，中文写作中不会使用这种字形，可以忽略。

有关公式与上下文间距的一些注意事项：请保证源码中的公式的环境（如 `\begin{equation}`）与上一段落不要有空行。否则，公式和上文段落之间会有额外的空白。

1.3 常见问题和疑难解答

如果您遇到生僻字无法显示、列表项间距过大、三线表需要跨页等问题，请参考[在线文档](#)的「疑难杂症」部分。

第 2 章 Franka 机械臂系统集成与 LeRobot 接口扩展

本章围绕毕业设计的主体工程工作展开，对 Franka 机械臂在实时控制侧的稳定运行、与 Hugging Face LeRobot 框架的对接方式，以及面向模仿学习数据采集所设计并开源的三维鼠标与同构主从臂两类遥操作方案进行系统论述。上述环节在逻辑上构成“底层执行—框架抽象—人机接口—数据闭环”的完整链条，为后续基于 $\pi_{0.5}$ 等策略模型的训练与桌面抓取实验提供硬件与数据基础。与同形态主从臂相比，单纯“关节到关节”的硬件搭建已被较多工作覆盖；本章在工程上的核心创新之一在于：在 LeRobot 统一抽象之下，将主从映射参数化为可文件化配置，并提供交互式两阶段配对标定脚本，用一次对齐姿态与一次夹爪开合行程，自动解算七轴偏置与夹爪量程，从而在装配误差、轴向反向与零点漂移存在时仍能稳定复现从臂指令，显著提升示教数据的可重复性与开源可复现程度。

2.1 系统总体架构

从系统分层角度，可将本文实现的机器人数据采集平台划分为四层。

1. **实时执行层**：部署在机器人实时控制机（RT PC）上的 `franka_server`，基于官方 `libfranka` 与轨迹平滑库 `Ruckig`，完成与真实 Franka 控制器的低层通信、关节空间运动生成与安全约束相关处理。
2. **网络服务层**：实时侧与控制计算侧通过 TCP 套接字交互，上层以文本行协议发送关节目标等指令并接收机器人状态反馈，从而将实时与非实时进程解耦。
3. **LeRobot 抽象层**：在 LeRobot 的 `Robot / RobotConfig` 约定下实现 `FrankaFER`（及带夹爪的复合机器人配置），统一观测（多相机 RGB/深度、关节状态、末端位姿等）与动作（关节目标等）的张量字典接口，使其可被 `lerobot-record`、重放与训练管线直接消费。
4. **遥操作与数据层**：在 `Teleoperator` 扩展上实现三维 `SpaceMouse` 笛卡尔增量映射与同构主从臂关节映射两类接口，并在 LeRobot 数据集规范下完成时间对齐与字段命名，支撑高质量示教采集。

表 2-1 汇总了主要软件模块与职责；表中以 **LeRobot 配置类型码** 标识实现入口，避免冗长 Python 类名挤占版心。

表 2-1 平台主要模块与功能对照（配置类型与中文对照）

模块	主要职责	配置 type / 可执行体
实时控制服务	实时控制、轨迹平滑、状态回传	<code>./franka_server</code> [IP]
Franka 从臂客户端	Socket 通信、观测/动作特征、连接管理	<code>franka_fer</code>
臂 + 夹爪从机	臂关节与夹爪指令/观测一体化	<code>franka_fer_gripper</code>
SpaceMouse 臂遥操作	鼠标增量—累积位姿—IK—关节目标	<code>franka_fer_spacemouse</code>
SpaceMouse 臂 + 爪	共用鼠标读数器，臂与爪动作对齐复合从机	<code>franka_fer_gripper_spacemouse</code>
同构主从（臂）	主臂读数缩放、偏置后映射为从臂关节目标	<code>franka_fer_subarm</code>
同构主从（臂 + 爪）	关节映射 + 主臂夹爪原始量归一化	<code>franka_fer_gripper_subarm</code>

说明：表中带 SpaceMouse 的遥操作对应实现模块分别位于 `src/lerobot/teleoperators/franka_fer_spacemouse/` 与 `src/lerobot/teleoperators/franka_fer_gripper_spacemouse/`；同构主从映射的核心类为 `FrankaFERSubarmTeleoperator` 与 `FrankaFERGripperSubarmTeleoperator`。下文在叙述中优先使用配置类型码指代，必要时再标明类名。

2.2 Franka 实时控制与底层通信

2.2.1 控制环路与时序性

Franka 机械臂的官方控制接口对实时性有严格要求。本文将低层控制集中于 RT-PC 上的 C++ 服务进程，避免在非实时操作系统上与 Python 解释器抢占同一调度上下文，从而降低控制超时风险。服务进程基于 `libfranka` 读取机器人状态并下发轨迹或目标；同时引入 `Ruckig` 对指令进行时间参数化和平滑处理，抑制示教或上层指令中的突变，提高执行过程的可重复性与安全性。

2.2.2 套接字协议与上下位机分工

上层 LeRobot 进程不直接调用 `libfranka`，而是通过 TCP 套接字与 `franka_server` 通信：连接建立后，以换行分隔的文本命令进行请求—应答式交互。该设计的优点在于：

- 上位机可使用与实验室计算环境一致的 Python 版本与依赖栈，便于与 LeRobot、PyTorch 等工具链共存；
- 实时机仅需维护体量可控的服务端构建产物与少量系统依赖，部署边界清晰；
- 便于在局域网内将相机与推理节点与机械臂分机关联，扩展多机协作时的拓扑。

启动数据采集或控制脚本前，须在 RT PC 上先启动 `franka_server` 并传入机器人 IP，确认健康检查通过后再在 LeRobot 侧实例化 `FrankaFER`，以避免“接口已初始化但执行端未就绪”的连接语义错误。

2.3 LeRobot 通用机器人接口扩展

2.3.1 框架约定与工厂注册

LeRobot 将物理平台抽象为 `Robot` 子类，并通过 `RobotConfig` 的数据类配置（含 `type` 字段）在工厂函数中派生具体实现。本文在 `make_robot_from_config` 中注册 `franka_fer`、`franka_fer_gripper` 等类型，使标准 CLI 与脚本能够以与其他官方机器人相同的方式构造 `Follower`，而无需在外部维护“专用启动器”。

2.3.2 FrankaFER 类设计要点

`FrankaFER` 的核心职责可概括为三方面。

（1）**连接与生命周期管理**。配置项中提供实时服务端 IP 与端口；实例化时建立阻塞或非阻塞套接字，并在断开、异常时清理文件描述符，向上层暴露 `is_connected` 等状态查询接口，便于录制脚本在断连时安全退出。

（2）**观测特征 `observation_features`**。对齐 LeRobot 数据集模式，显式声明各观测键名及类型：七自由度关节位置与速度、末端执行器位姿（以行优先 4×4 齐次变换矩阵展平为十六维向量）、以及按相机名称组织的 RGB 张量；若启用 `RealSense`

等相机的深度流，则同步声明对应深度图键名与形状。该声明在录制时用于分配缓冲区与校验字段完备性。

(3) 动作特征 `action_features` 与下发。本文在标准臂控制模式下将动作空间取为七维关节目标位置，键名与 LeRobot 惯用命名一致，便于与后续策略头维匹配。发送时在语义上执行“将一行命令编码并写入套接字—读取应答”的同步原语，并对超时与异常进行日志记录与重连策略预留。

上述设计使 Franka 平台在 LeRobot 语义下与 Koch、SO-100 等 follower 等价：同一套 record/replay 流程可以迁移使用，仅更换配置 JSON 或 Draccus 参数即可。

2.3.3 带末端夹爪的复合机器人

针对桌面抓取任务，机械臂需配合二指夹爪等末端执行器。本文在 FrankaFERGripper 中将臂部与夹爪作为组合实体建模：观测维度在关节与末端位姿基础上扩展夹爪开合相关量；动作维度在七维关节之外增加归一化到 $[0, 1]$ 的夹爪指令通道。由此，单条时间步上的“状态—动作”对与单一末端执行器语义一致，降低多设备拼接时的字段分叉与后处理成本。

2.4 三维鼠标遥操作方案

2.4.1 硬件与数据获取

三维鼠标（SpaceMouse 等六自由度输入设备）能够以较高频率输出线性与角增量的组合。本文在 LeRobot 依赖中采用 `pyspacemouse` 读数，由 `SpaceMouseStateReader` 在独立线程或上下文管理器中循环采样，将 USB HID 事件转为统一的增量序列号与数值结构，供上层遥操作类消费。

2.4.2 累积位姿与死区处理

直接将单次增量映射为关节增量易导致漂移与抖动。本文实现 `DeltaPoseAdapter`，在笛卡尔空间维护累积位置向量与累积四元数姿态：对每个采样周期，将设备上报的平移增量按可配置标尺缩放，对平移范数与旋转角施加死区，低于阈值的增量置零；合法增量通过四元数乘法累积到当前姿态。序列号发生变化时才更新累积状态，避免同一硬件包被重复积分。

设第 k 次有效采样对应的平移增量为 $\Delta \mathbf{p}_k \in \mathbb{R}^3$ ，姿态增量为单位四元数 $\Delta \mathbf{q}_k$

（采用 (x, y, z, w) 约定），则累积位姿更新可写为

$$\mathbf{p}_{k+1} = \mathbf{p}_k + s \cdot \Delta \mathbf{p}_k, \quad \mathbf{q}_{k+1} = \text{normalize}(\mathbf{q}_k \otimes \Delta \mathbf{q}_k), \quad (2-1)$$

其中 s 为位置缩放系数， $\otimes$ 表示四元数乘法。死区在平移范数与旋转角上已并入 $\Delta \mathbf{p}_k$ 、 $\Delta \mathbf{q}_k$ 的筛选逻辑。

2.4.3 逆运动学与 Teleoperator 接口

在获得期望腕部位姿后，需解算七轴关节目标。本文复用与 VR 遥操作一致的 ArmIKProcessor 路径，在 C++ 桥接与 Python 包装之间传递参考关节与目标位姿，对奇异位形与求解失败做异常捕获与日志输出，避免单次 IK 失败中断整条录制会话。

配置类型 `franka_fer_spacemouse` 对应的遥操作类继承 LeRobot 的 Teleoperator，实现 `action_features`、`connect/disconnect`、`get_action` 等契约：每次 `get_action` 将当前鼠标增量经适配器积分后送 IK，输出七维关节目标字典，键名与 FrankaFER 的动作键一致，从而实现“人通过鼠标在笛卡尔空间表达意图—机器人在关节空间跟随”的闭环。

臂 + 夹爪组合。配置类型 `franka_fer_gripper_spacemouse` 在臂部通道上与上述实现共享同一 `SpaceMouseStateReader`，避免对同一 USB 资源双开；夹爪通道由配置映射为开合标量，与臂部动作合并为复合动作字典，便于与 `franka_fer_gripper` 联合录制。

2.5 同构主从臂遥操作、配对标定与数据采集

同构主从方案是本文面向桌面抓取示教的数据采集主路径之一：操作者拖动与 Franka 同结构的主臂（代码中以 `SoFranka` 抽象底层总线读数），从臂实时跟踪，可在关节空间完成贴近台面、绕行障碍等细调。与仅开源硬件清单不同，本文的可复现性体现在：（1）映射公式与运行时配置字段一一对应；（2）提供参数自动解算工具，把“肉眼对齐主从”转化为可写入 JSON 的数值，避免手工试凑比例与零点。

2.5.1 同构构型的工程动机

相比三维鼠标等笛空间设备，同构主从允许示教式关节空间映射：主从几何相似时，操作者对主臂的关节角直觉可迁移到对从臂运动的预期，对抓取流水线中的姿态调整往往更直接。工程上，主臂与从臂的安装轴向、零点与传动比未必一致，因

此必须在软件层引入每关节缩放与偏置，并在夹爪通道引入原始量到 $[0, 1]$ 的线性标定；若缺少系统化标定，示教轨迹会在回放时出现整体偏转、夹爪“开不满/关不严”等问题，直接污染模仿学习数据集。

2.5.2 关节空间映射模型

主臂读出的七关节量经缩放与偏置后作为从臂关节指令。设主臂第 i 关节原始读数为 θ_i^L ，经模式函数 $\phi(\cdot)$ 转为主臂侧用于映射的弧度量 $\tilde{q}_i^L$ （例如度模式时 $\tilde{q}_i^L = \text{rad}(\theta_i^L)$ ；规范化模式时 ϕ 为恒等映射并保持与运行时一致）。记缩放 α_i 、偏置 β_i （单位：rad），则从臂指令为

$$q_i^F = \alpha_i \tilde{q}_i^L + \beta_i, \quad i = 1, \dots, 7. \quad (2-2)$$

其中 α 不仅承担比例尺度，更常取 ± 1 以修正轴向反向； β 承担零点对齐。式 (2-2) 与配置类型 `franka_fer_subarm` 的实现一致；在臂+爪配置 `franka_fer_gripper_subarm` 中，七轴仍遵循式 (2-2)，夹爪另述。

2.5.3 两阶段配对标定与自动修正

为从一次对齐姿态与一次夹爪行程中解出 α 、 β 及夹爪端点，本文实现脚本 `scripts/franka_gripper/calibrate_subarm_pairing.py`（下称配对标定脚本）。脚本在交互上分为两步，输出可直接粘贴至 `FrankaFERSubarmTeleoperatorConfig` 或 `FrankaFERGripperSubarmTeleoperatorConfig` 的 JSON 片段。

(1) 关节对齐阶段（解算偏置向量）。操作者与控制员将主臂与 Franka 摆到同一物理姿态（建议选用重复性高的 `home` 或示教起点）。脚本通过 `SoFranka` 读取主臂七关节当前值，按运行时计划使用的度/规范化模式转为 $\tilde{q}_i^L$ ；同时提示输入 Franka 该姿态下的七关节角（缺省为弧度输入，亦可用 `--franka-input-degrees` 以度制录入后内部换算）。缩放向量 α 由命令行 `--joint-scale` 给出（七个逗号分隔浮点数，典型为 ± 1 组合，用于在安装反向后仍能逐轴对齐）。偏置自动修正公式为

$$\beta_i = q_i^F - \alpha_i \tilde{q}_i^L, \quad i = 1, \dots, 7, \quad (2-3)$$

其中 q_i^F 为 Franka 在该对齐姿态的关节弧度。式 (2-3) 与 `FrankaFERSubarmTeleoperator` 中“先按 α 缩放主臂量、再加 β ”的语义严格一致，从而保证标定与运行时同一解。若主臂工作于度模式而 Franka 始终以弧度为指令单位，脚本在文档与日志中明确该

换算顺序，避免“混用物理单位导致偏置错一个 π ”的常见失误。

（2）夹爪量程阶段（解算开合端点）。在完成或不参与臂阶段标定后，操作者将主臂夹爪完全闭合与完全张开各一次并在提示时确认；脚本记录两次原始读数 g_{closed} 、 g_{open} ，对应写入配置的 `gripper_raw_at_closed`、`gripper_raw_at_open`。运行时归一化与录制脚本中一致：

$$g_{[0,1]} = \text{clip}\left(\frac{g_{\text{raw}} - g_{\text{closed}}}{g_{\text{open}} - g_{\text{closed}}}, 0, 1\right), \quad (2-4)$$

并在 $g_{\text{open}} \approx g_{\text{closed}}$ 等退化情形保留安全截断逻辑。若开闭原始读数方向相反，运行时仍可通过交换端点定义或反向映射保持 0 对应闭合、1 对应张开（与 `FrankaFER-GripperSubarmTeleoperator` 中的分支一致）。

工程选项与可维护性。脚本支持 `--leader-port` 指定主臂串口；`--leader-normalized` 切换主臂关节是否为 $[-100, 100]$ 规范化读数而非度；`--skip-arm` / `--skip-gripper` 仅重标定其中一环，便于现场迭代。标定结束后脚本以 JSON 统一打印 `arm` 与 `gripper` 字段及注释字段，可直接进入版本管理，实现“更换主臂/重新固零—重跑脚本—更新配置文件”的闭环。

2.5.4 与录制管线及数据质量的衔接

配置类型 `franka_fer_subarm` 与 `franka_fer_gripper_subarm` 均实现统一 `get_action`，可与 LeRobot 标准录制流程或自定义循环组合：交替读取机器人观测与遥操作动作，按 LeRobot 数据集 `schema` 序列化。时间戳对齐上，建议以机器人状态回执或统一主机时钟为基准，对图像帧与关节指令做最近邻或插值对齐；工程上可通过固定控制频率与阻塞读减少抖动，并在元数据中记录相机帧率与伺服周期。经过配对标定后的主从映射，可显著降低“同一条示教意图”在关节空间的批间方差，有利于小规模高质量轨迹（例如五十条量级）上策略网络的收敛与泛化——该数据规模与后续 $\pi_{0.5}$ 实验章节相对应。

2.6 开源与可复现性说明

本文将上述扩展以 Git 仓库形式公开发布（基于 LeRobot 指定 `commit` 的补丁式目录合并），包含 `franka_server` 构建脚本、Python 侧 `src/lerobot` 增量、`scripts` 下 `SpaceMouse`/主从臂/夹爪组合的录制与回放示例，以及配对标定脚本 `scri`

pts/franka_gripper/calibrate_subarm_pairing.py。第三方使用流程可概括为：在 RT PC 编译并常驻服务端；将 Python 包安装为可编辑模式；按硬件拓扑选择 RobotConfig/TeleoperatorConfig 的 type 并填入网络与串口参数；运行配对标定脚本生成 JSON；最后调用 LeRobot 数据录制接口完成示教。

通过该方式，Franka 不但在物理意义上“控制稳定”，在软件生态上与 LeRobot 的“通用机器人与遥操作抽象”对齐；同构主从路径在**参数可标定、可版本化**意义上形成可独立复用的工程模块——这也是本章相对“单纯驱动打通”而言的主要贡献。

第3章 桌面抓取示教数据采集与 LeRobot 数据集规范

第二章从工程实现角度完成了 Franka 与 LeRobot 的接口打通，并给出了三维鼠标与同构主从两类遥操作方案及其标定流程。本章在此基础上，以实际落地的一条示教采集流水线为对象，说明采集任务设定、落盘数据如何组织为 LeRobot 官方数据集形态（codebase_version v2.1），以及为何该字段编排与多相机布局适合本文的桌面抓取与后续 $\pi_{0.5}$ 类策略训练。文中以仓库内数据集目录 pick_apple_4_18/ 为实例——该数据集即毕业设计录制并用于 $\pi_{0.5}$ 微调实验的抓取示教数据。

3.1 采集任务与录制流程概要

语言指令层面的任务语义单独登记在 pick_apple_4_18/meta/tasks.jsonl 中：任务索引 task_index=0 对应自然语言描述

```
Pick up the green apple and place it in the upper left corner  
of the table.
```

每条 Episode 在 meta/episodes.jsonl 中记录所属任务索引、自然语言列表及本条轨迹的时间步长度；全库共 30 条 Episode，可与第二章所述 lerobot-record 流程一一对应（先完成机器人与相机就绪，再在同一任务语义下重复示教，最后由 LeRobot 按统一 schema 写盘）。

环境与控制侧要点沿用第二章的分层架构：低频训练侧仍以 LeRobot Robot/Teleoperator 抽象消费观测与生成动作指令，高频实时控制由 RT PC 上的 franka_server 执行。录制时帧率与时间戳对齐在元信息层面体现为统一的标称帧率 10 Hz（下文 meta/info.json 中 fps 字段），对桌面抓取这类以准静态接触与柔顺放置为主的操作而言，相较于单纯追求高频采样，更看重示教动作的连贯性、观测与动作的锁相以及多相机覆盖；本文选择 10 Hz 作为视频编码、parquet 行频与伺服周期的共同基准，在保证任务相关运动信息不丢失的前提下，降低存储与解码开销，并与 LeRobot 训练数据加载器的常见假设兼容。

3.2 LeRobot v2.1 目录结构与元数据

数据集 `pick_apple_4_18` 的顶层结构遵循 LeRobot2.x 数据集规范,可被 `lerobot.datasets` 直接按 Repo ID 或本地路径打开。其核心组成可概括为四类路径（以下均相对于数据集根目录）:

1. **meta/**——机器可读的全局说明与 Episode 列表: `info.json` 汇总机器人类型、总 Episode 数、总帧数、视频与表格路径模板及各字段特征（名称、`dtype`、`shape`）; `tasks.jsonl`、`episodes.jsonl`、`episodes\_stats.jsonl` 分别描述任务条目、每条轨迹的语义标签与长度、以及轨迹级简易统计。
2. **data/**——按 chunk 存放的帧级表格（本数据集仅占一个 chunk）: `data/chunk-000/episode_XXXXXX.parquet`, 每文件对应一整条 Episode 的时间序列样本。
3. **videos/**——与表格行对齐编码的外挂视频轨道: `videos/chunk-000/<video\_key>/episode_XXXXXX.mp4`。 `info.json` 中的 `video_path` 模板指明 `video_key` 由特征键推导（例如下文两个 RGB 相机的键名即对应两级子目录）。
4. （可选的其他辅助文件可按 LeRobot Hub 惯例存在；本文数据集以精简可训练闭环为原则，不包含与训练无关的中间产物。）

`meta/info.json` 中与本实验直接相关的全局量如下:

- 机器人标识: `robot_type` 声明为 FrankaFER 末端带二指夹爪的组合物 (`franka_fer_gripper`) 与同路径下的观测/动作八维向量一致;
- 规模概览: `total_episodes=30`, `total_frames=5512`, `total_videos=60`（每条 Episode、每个相机各一条视频）; `splits.train` 为 0:30, 即当前全部数据用于训练集划分;
- 路径模板: `data_path` 与 `video_path` 使用 LeRobot 统一的格式化占位符, 将逻辑 Episode 索引映射到具体文件名, 便于分块扩展与大规模数据管理。

该布局的价值在于: 表格存低维时序量、视频存高维图像流, 二者通过同一套 `frame_index/episode_index/时间戳` 字段建立一一对应关系; 训练代码只需按 LeRobot 的 `features` 声明进行张量拼接, 无需为 Franka 定制私有 `DataLoader`。

3.3 帧级特征模式与模仿学习接口

表 3-1 归纳了 `info.json` 中声明的主要特征键。可以看到，动作空间与关节观测在维数与命名上呈对齐关系：均为八维浮点向量，前七维对应 Franka 七轴关节位置，第八维为夹爪开合通道（与第二章 FrankaFERGripper 约定一致）。这种“状态—动作同构”的写法是行为克隆与扩散策略类方法的常见输入形式：网络在时刻 t 读取多相机图像与本体状态，预测监督目标为同刻示教者给出的关节—夹爪指令或下一时刻目标。

表 3-1 数据集 `pick_apple_4_18` 的主要帧级特征（摘自 `meta/info.json`）

特征键	dtype	形状概要	含义（本文设置下）
<code>action</code>	<code>float32</code>	8	示教关节目标 + 夹爪指令，用于监督学习
<code>observation.state</code>	<code>float32</code>	8	关节位置与夹爪反馈，构造策略条件
<code>observation.velocity</code>	<code>float32</code>	7	七轴关节角速度，可选辅助信号
<code>observation.eepose</code>	<code>float32</code>	7	末端位姿 ($x, y, z, q_x, q_y, q_z, q_w$)
<code>observation.images.wrist</code>	视频	720×1280×3	腕部/手眼附近 RGB，AV1，10 fps
<code>observation.images.third_person</code>	视频	720×1280×3	第三人称场景 RGB，AV1，10 fps
<code>timestamp</code> 等索引字段	标量	1	帧内时间戳、Episode/任务编号、全局行号

多相机观测分别以 `observation.images.wrist` 与 `observation.images.third_person` 两个键挂载。二者在物理视角上互补：第三人称摄像机提供桌面上苹果目标物、障碍物与边角放置区域的全景关系，适于粗定位与语义对齐；腕部摄像机则提供抓取对准期所需的局部几何与开合过程中的手指—物体相对关系，对降低从侧面观看时的深度歧义至关重要。分辨率 1280×720、采样率与关节序列一致且视频参数在 `features.info` 块中逐项声明（像素格式 `yuv420p`、是否具有深度、是否含音频等），保证了跨机器解码的可重复性。

低维本体量除关节状态外，另含七维关节速度与七维末端位姿。前者可用于未来

引入速度惩罚或对比学习；后者便于在调试阶段与笛卡尔空间示教设备（如第二章 SpaceMouse 路径）进行交叉验证。对于以关节空间监督为主的 $\pi_{0.5}$ 管线，核心训练张量仍自然落在 `observation.images.*` 与 `action`（及必要的 `observation.state`）之上，其余字段以“可扩展但不扰动训练接口”的方式存在。

3.4 数据规模、质量与后续训练衔接

在规模上，30 条 Episode、合计 5512 帧属于小规模专用数据集，适用于在基础视觉—语言—动作（VLA）模型上进行任务特化微调或概念验证，而不追求覆盖所有桌面扰动。单条 Episode 长度在约一百七至一百九十余步之间波动，对应十几秒量级的操作过程，与“抓取—移动—放置”三阶段结构相符。

从数据质量角度，本文在采集端强调三点，并由此体现为落盘格式上的优势：（1）**字段完备性**——关键键名符合 LeRobot 约定，减少训练脚本中的硬编码映射；（2）**时间一致性**——统一 10 Hz 与外挂视频，使视觉与关节标签天然对齐；（3）**任务语义显式化**——自然语言任务写入 `tasks.jsonl` 并在 Episode 元数据中重复索引，便于多任务扩展或与语言条件策略对接。在此前提下，即使样本量有限，网络仍主要学习桌面绿色苹果抓取并置于桌面左上区域这一窄分布，与后续章节对 $\pi_{0.5}$ 结构的针对性修改及桌面实验评价形成闭环。

3.5 基于 Rerun 的可视化核验

录制完成后，使用 Rerun 等时序可视化工具对 `pick_apple_4_18` 进行抽检，可并行查看多路图像与低维曲线，快速发现丢帧、相机未触发、动作饱和等异常。图 3-1 展示了典型一帧附近的多模态观测：腕部与第三人称两路 RGB、七关节位置与七维末端位姿随时间的变化曲线。可见关节与末端量随操作阶段连续变化，无阶跃式异常；图像内容与桌面抓取场景一致。该步骤并不改变数据集文件结构，但为“格式正确 $\neq$ 语义正确”提供了直观审查手段，与 `episodes\_stats.jsonl` 中的数值范围统计互为补充。

3.6 本章小结

本章以 `pick_apple_4_18` 为实例，说明了本文桌面抓取示教数据在 LeRobot v2.1 规范下的目录组织、元数据文件、帧级特征键及双相机视频布局，并交代了数

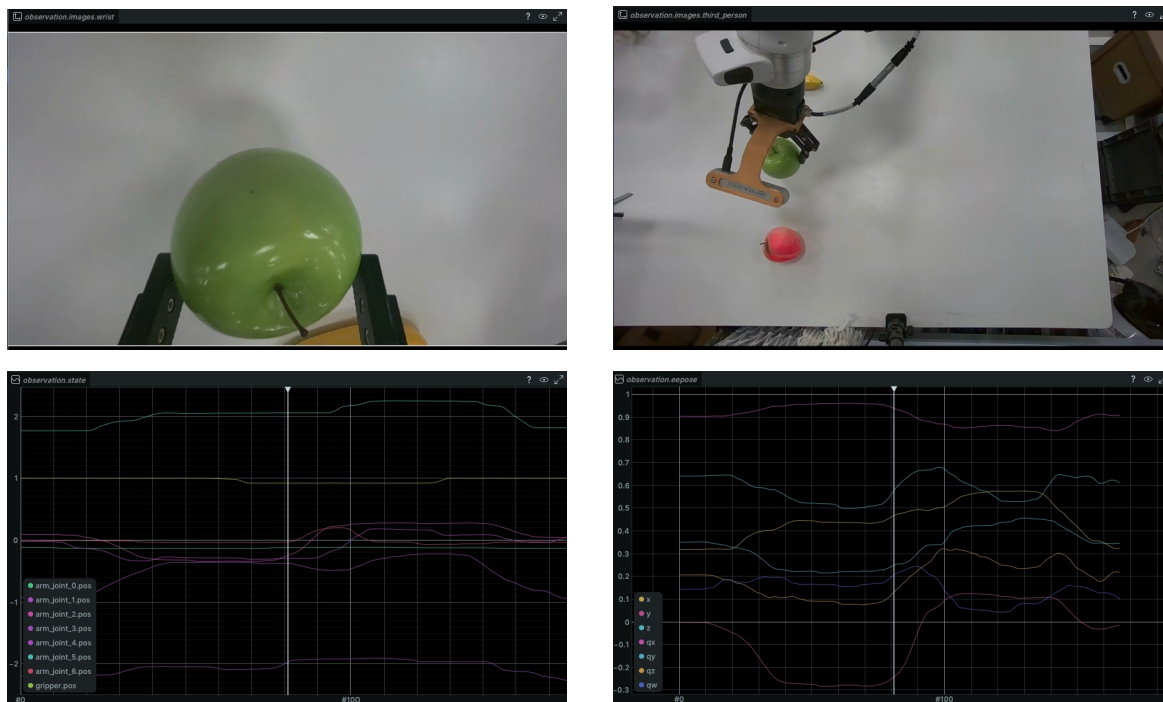


图 3-1 抓取示教数据集 `pick_apple_4_18` 在 Rerun 中的多模态观测示例。上方为腕部相机与第三人称相机画面；下方为七关节位置与末端位姿（位置与四元数分量）随时间的曲线。

据规模与面向 $\pi_{0.5}$ 微调的使用方式。该数据集在字段层面与官方生态对齐，在内容层面覆盖“第三人称场景理解 + 腕部精细对准 + 关节/夹爪动作监督”，为后续策略训练与实机实验提供了可直接加载的标准化输入。

第4章 基于 $\pi_{0.5}$ 的桌面抓取策略训练与实验分析

前两章分别完成了 Franka 机械臂与 LeRobot 通用接口的打通，以及桌面抓取示教数据的标准化采集与组织。本章在此基础上，围绕本文的算法侧工作展开，重点说明：其一， $\pi_{0.5}$ 模型[3]的核心原理与为何其适合迁移到桌面抓取任务；其二，本文如何将 Franka 双视角观测、八维状态/动作以及自然语言任务提示映射到 OpenPI 的训练接口，并基于自采的 pick_apple_4_18 数据集完成微调训练；其三，训练后模型在不同在线执行方式下呈现出的显著性能差异，尤其是逐段闭环推理带来的成功率提升；其四，结合本文改动与实验观察，对若干模型变体给出定性分析与后续优化方向。

需要指出的是，本文的算法贡献并非简单复现已有 $\pi_{0.5}$ 结果，而是将其从原论文面向移动双臂家庭机器人、长时程多阶段任务的设定，迁移到**静态 Franka 桌面抓取**这一更聚焦但接触精度要求更高的场景中。为此，本文完成了**数据接口重构、训练配置适配、在线执行闭环设计与行为层面的现象分析**，从而形成“硬件接入—示教采集—策略训练—实机部署”的完整毕业设计闭环。

4.1 $\pi_{0.5}$ 的核心原理框架

4.1.1 从 VLA 到分层推理

视觉—语言—动作模型（Vision-Language-Action, VLA）试图直接学习从观测与语言指令到机器人动作的映射。对单步或动作块预测而言，其基本目标可写为

$$\max_{\theta} \mathbb{E}_{(\mathbf{a}_{t:t+H}, o_t, \ell) \sim \mathcal{D}} \log \pi_{\theta}(\mathbf{a}_{t:t+H} \mid o_t, \ell), \quad (4-1)$$

其中 o_t 包含时刻 t 的多相机图像与机器人本体状态， ℓ 为自然语言任务指令， $\mathbf{a}_{t:t+H}$ 表示长度为 H 的动作块。与传统仅依赖低维状态的行为克隆不同，VLA 同时利用视觉与语言信息，使策略具备更强的任务条件表达能力与跨对象泛化潜力[4-5]。

$\pi_{0.5}$ 的关键思想在于：**将高层语义推理与低层动作生成统一到同一个多模态序列建模框架中**，而不是单纯把“图像 + 文本”直接映射为动作。传统的端到端模仿学习策略往往直接输出控制量，虽然在单一任务上可以工作，但一旦任务中包含“先观察目标、再对准、再夹取、再搬运”的多阶段结构，纯低层策略往往难以显式表达

当前子目标。 $\pi_{0.5}$ 试图解决的正是这一问题：它既保持了 VLA 对视觉和语言的理解能力，又把“当前应该做什么”编码成显式语义中间层，使动作生成不再完全依赖隐式内部状态。

原论文将模型分解为高层子任务推断与低层动作预测两部分，其联合分布可写为

$$\pi_{\theta}(\mathbf{a}_{t:t+H}, \hat{\ell} \mid o_t, \ell) = \pi_{\theta}(\hat{\ell} \mid o_t, \ell) \pi_{\theta}(\mathbf{a}_{t:t+H} \mid o_t, \hat{\ell}), \quad (4-2)$$

其中 $\hat{\ell}$ 表示模型根据当前场景与高层任务自动生成的语义子任务，例如“抓住苹果”“移动到桌面左上角”等。该结构的意义在于：模型可以先决定“当前最合适做什么”，再决定“具体如何做”，从而把长时程任务分解成若干更稳定的局部动作过程。这种思路与语言模型中的链式推理存在相似性，但 $\pi_{0.5}$ 将其落到了机器人可执行的动作闭环中[3]。从控制角度看，式 (4-2) 也意味着模型把任务分解误差与动作跟踪误差在建模层面分离开来：高层语义负责减少“做错事”的风险，低层动作负责减少“做不准”的风险。

对于本文的桌面苹果抓取任务，虽然任务长度远短于原论文中的“整理厨房”“铺床”等家庭长任务，但接近目标、夹取接触、抬升确认、放置到指定区域依然天然构成了一个多阶段操作链。因此， $\pi_{0.5}$ 的分层思想依旧具有价值：高层语义可帮助策略在“寻找目标-对准抓取-完成搬运”之间切换，而低层动作块则负责生成连续、可执行的关节与夹爪指令。特别是在苹果这类易滚动、易滑移物体的抓取中，若没有对子目标的良好组织，低层策略很容易把“接近目标”和“稳定夹持”混为一个不可分的黑盒过程，导致接触阶段误差被快速放大。

4.1.2 统一多模态序列建模

$\pi_{0.5}$ 的第二个关键点，是它并没有为“看图”“理解语言”“预测子任务”“输出动作”分别设计彼此割裂的模型，而是试图把这些信息都放到一个统一的序列建模框架中。其底层做法是：将图像 patch、文本 token、机器人本体状态以及动作相关表示映射到同一 Transformer 中，通过注意力机制建立跨模态关联。这样一来，模型在看到第三人称图像中的苹果、腕部相机中的夹爪位置以及文本提示“抓取苹果并放到左上角”时，可以在统一特征空间里决定当前最相关的信息组合。

从原论文和本文本地实现看， $\pi_{0.5}$ 并不是简单把所有 token 串在一起做标准因果

注意力，而是使用了前缀-后缀式的注意力组织。图像、语言提示以及部分状态信息作为前缀，允许在前缀内部进行充分的信息交互；动作相关 token 则作为后缀，在访问前缀条件信息的同时保持适合动作生成的依赖结构。这样的设计有两个直接好处：

1. **充分利用视觉语义上下文。**动作生成不再只依赖某一个单独编码后的观测向量，而是依赖整个多模态上下文序列；
2. **便于统一高层与低层推理。**同一个主干既可以输出文本型子任务，也可以为后续动作专家提供条件特征。

对本文任务而言，这种统一建模尤其重要。由于桌面抓取同时依赖全局视角与局部手眼视角，如果采用两套完全分离的感知链路，系统往往会面临信息对齐复杂、训练目标割裂的问题。而在 $\pi_{0.5}$ 框架中，第三人称图像更偏向提供目标位置和放置区域的全局语义，腕部图像更偏向提供夹取局部几何与接触反馈，两者都被放入同一个条件上下文中，因此更适合学习“先粗定位、再精对准”的策略模式。

4.1.3 主干网络与动作专家

在结构上， $\pi_{0.5}$ 建立在 π_0 [4] 之上，延续了“视觉语言主干 + 动作专家（action expert）”的设计。主干部分继承预训练视觉语言模型的表征能力，其核心由 SigLIP 视觉编码器和 Gemma 语言模型骨干组成；图像被切分为 patch token，自然语言指令被编码为文本 token，而机器人状态与动作则通过专门的投影与编码方式进入同一 Transformer 序列中[6]。这样做的本质，是把机器人控制问题转化为统一的多模态序列建模问题。

与纯文本自回归模型不同，机器人策略不仅要输出语义 token，还要输出连续动作。为此， $\pi_{0.5}$ 在主干之外引入更小的动作专家模块，对动作块进行专门建模。动作专家的作用可以概括为两点：

1. **分离语义推理与连续控制。**主干网络保留语言与视觉理解能力，动作专家则专注于生成高精度、低时延的连续控制量。
2. **提高在线推理效率。**若直接以离散动作 token 的自回归方式逐个采样，实机部署的时延较大；动作专家将动作块视为连续变量分布，能以更少的推理步数生成整个动作序列。

相比之下， $\pi_{0.5}$ 相对于 π_0 有两处重要改变：其一，机器人状态可以作为离散语言 token 的一部分输入，而非仅作为连续后缀输入；其二，在动作专家中使用自适应归一化的方式注入 flow matching 时间步。这一点与本文本地代码中的实现是吻合的：当训练配置设置 `pi05=True` 时，`pi0_config.py` 会将模型类型切换到 PI05，并启用“离散状态输入 + adaRMSNORM 时间注入”的路径。这说明本文并非只是在配置层面重命名模型，而是确实采用了接近论文原意的 $\pi_{0.5}$ 结构。

4.1.4 动作专家中的 Flow Matching 机制

动作专家与 flow matching 的结合，是 Pi 系列模型区别于许多传统 VLA 的核心创新之一。其出发点在于：离散 token 对大规模预训练很友好，但真实机器人控制需要的是连续、平滑、低延迟的动作输出；如果完全依靠离散动作 token 自回归采样，不仅推理速度受限，而且连续控制精度也可能受动作量化误差影响。Pi 系列的关键设计，就是在保留大模型主干语义能力的同时，用一个更小、更专门化的动作专家来建模连续动作分布[3-4]。

设真实动作块为 $\mathbf{a}_{t:t+H}$ ，噪声为 $\boldsymbol{\omega} \sim \mathcal{N}(0, I)$ ，时间变量为 $\tau \in [0, 1]$ ，则 flow matching 中的中间变量可写为

$$\mathbf{x}_\tau = \tau \boldsymbol{\omega} + (1 - \tau) \mathbf{a}_{t:t+H}. \quad (4-3)$$

其直观含义是：当 τ 接近 1 时， $\mathbf{x}_\tau$ 更接近纯噪声；当 τ 接近 0 时， $\mathbf{x}_\tau$ 更接近真实动作。动作专家并不直接回归最终动作，而是学习一个向量场，把当前的噪声化动作逐步“拉回”到真实动作块所在的分布流形上。若记目标向量场为

$$\mathbf{u}_\tau = \boldsymbol{\omega} - \mathbf{a}_{t:t+H}, \quad (4-4)$$

则模型的任务可以理解为：在给定观测条件 o_t 、任务提示 ℓ 以及当前中间状态 $\mathbf{x}_\tau$ 的情况下，预测一个足以将当前样本推向真实动作的连续更新方向。

这种设计相比直接回归动作块，具有三方面优势。第一，**表达能力更强**。对于多峰或接触敏感的动作分布，直接回归往往倾向于输出平均动作，而 flow matching 更适合刻画连续动作分布的形状。第二，**推理效率更高**。模型不需要逐 token 自回归采样完整动作序列，而是通过少量迭代积分生成整段动作。第三，**与大模型主干天然解耦**。动作专家可以比主干小很多，只负责连续控制这一件事，因此既保留了主干

的视觉语言能力，又避免把所有控制负担都压到超大参数主干上。

Pi 系列的一个非常关键的工程点，是如何把时间步 τ 注入动作专家。 π_0 主要通过将时间信息与动作 token 混合后再输入动作分支；而 $\pi_{0.5}$ 更进一步，采用自适应归一化的方式注入时间条件，使每一层动作专家都能感知当前的去噪阶段。对应到本文本地实现，pi0.py 中当 pi05=True 时会启用独立的时间 MLP，并将时间嵌入作为 adaRMSNORM 的条件输入，而不是简单拼接到动作 token 后。这一改动的好处在于：时间步信息对整条去噪轨迹的调制更加稳定，也更符合“动作专家专门负责连续流场生成”的设计目标。

从本文桌面抓取任务看，flow matching 的价值尤其体现在接近-闭合-抬升这类短时连续接触动作中。若动作输出过于离散，夹爪闭合时机、抬升速度和末端姿态变化都容易产生不连贯现象；而 flow matching 生成的连续动作块更容易保持轨迹平滑。进一步结合本文后续的分段闭环执行策略，可以把这种连续动作块理解为“局部时间窗内的平滑控制建议”，再通过频繁重推理不断修正，从而兼顾了平滑性与反馈性。

4.1.5 离散动作预训练与连续动作后训练

$\pi_{0.5}$ 训练流程的第二个核心，是离散动作表示与连续动作表示的结合。原论文在预训练阶段使用 FAST 动作离散化[7]，将动作块编码为离散 token，从而能够用标准自回归 next-token prediction 的方式大规模高效训练；而在后训练阶段，则引入 flow matching[8] 风格的连续动作生成机制，以获得更适合实时控制的连续输出。更具体地说， $\pi_{0.5}$ 不是在预训练阶段就完全依赖连续动作建模，而是先利用离散动作 token 获得高效、稳定的大规模训练，再在后训练阶段加入动作专家与连续动作目标。这样做的原因是：离散 token 训练与大模型生态更加兼容，便于复用已有视觉语言主干；而连续动作建模则更适合最终实机部署。原论文形式化地将总目标写成文本交叉熵损失与连续动作场回归损失的加权和[3]。对本文而言，这一机制的意义在于：

1. 训练阶段能够继承预训练 VLM 在语义与视觉上的能力；
2. 部署阶段能够直接输出连续关节/夹爪动作块，避免全自回归动作采样导致的时延；
3. 通过动作块预测，使控制器在短时域内保持动作连续性与平滑性。

4.1.6 异构数据协同训练的意义

$\pi_{0.5}$ 之所以区别于传统桌面行为克隆策略，不只因为模型结构更大，更重要的是其协同训练（co-training）思想。原论文并不是只用目标机器人自己的示教数据训练策略，而是把多种来源的数据放进同一个训练食谱中，包括：不同机器人本体的数据、跨场景的数据、高层语义子任务标注、网页图文数据以及目标平台上的后训练数据[3]。这种思路的直接收益是：即便目标场景上的机器人数据规模有限，模型依然可以从外部多模态知识中继承对象识别、语言对齐与任务分解能力。

本文的桌面抓取实验只使用了 30 条自采轨迹，规模远小于原论文数百小时级别的数据量，因此更能体现 $\pi_{0.5}$ 这种“先借助大模型与异构数据获得通用能力，再用小规模专用数据做任务特化”的价值。换言之，本文并不是试图从零训练一个桌面抓取策略，而是将一个具有较强视觉语义先验的 VLA 基座，通过 Franka 实验数据快速迁移到苹果抓取这一具体任务中。

4.2 面向 Franka 桌面抓取的模型适配与训练实现

4.2.1 任务观测与动作空间重定义

原始 $\pi_{0.5}$ 论文面向带移动底盘、升降机构与双臂夹爪的家庭服务机器人，其状态与动作空间维数达到 18 ~ 19 维，且在高层推理时会使用四路相机，在低层动作推理时使用前向相机与双腕相机[3]。本文场景则聚焦于静态 Franka 单臂桌面抓取，其控制对象与观测形式更紧凑，因而必须进行接口层与数据层的重新定义。

根据本文训练配置脚本 config.py 中的 pi05_franka_pick_apple 任务，训练数据被重排为如下结构：

- **图像输入：**将第三人称相机 observation.images.third_person 映射为 images/base，将腕部相机 observation.images.wrist 映射为 images/wrist；
- **状态输入：**使用 observation.state 作为低维本体状态；
- **动作监督：**使用 action 作为策略目标；
- **语言提示：**使用数据集中的 prompt 字段作为任务文本输入。

结合第三章的数据集定义可知，本文任务中的 state 与 action 均为八维向量：前七维对应 Franka 七轴关节，第八维对应夹爪开合量。这样一来，本文的动作空间

既保留了夹取任务所需的末端开合信息，又避免了在小数据场景下额外建模底盘与多臂协调，从而使训练重点聚焦到“桌面目标定位、抓取对准、夹紧抬升与搬运放置”这几个关键环节。

这种改写的价值在于：虽然本文采用的是 $\pi_{0.5}$ 的总体思路，但并未机械照搬原论文的机器人输入格式，而是将 **Franka** 的观测-动作接口完全纳入 **OpenPI** 可消费的通用数据范式。这正是第二章与第三章工作的算法侧落点：没有前面的接口统一与数据规范化，就无法把原本服务于通用机器人 VLA 的训练框架直接用于 **Franka** 实机任务。

4.2.2 训练配置与关键超参数

表 4-1 汇总了本文桌面苹果抓取任务的主要训练设置。可以看到，训练配置整体遵循“加载 $\pi_{0.5}$ 基础权重，在本地数据集上进行任务微调”的思路，其中既保留了原模型动作块预测的优点，也针对桌面任务做了更紧凑的时域设置。

表 4-1 本文 $\pi_{0.5}$ Franka 苹果抓取实验的主要训练配置

配置项	取值与说明
任务名 / 工程名	pi05_franka_pick_apple
模型配置	Pi0Config(action_horizon=32, max_token_len=200, pi05=True)
数据集	repo_id=["pick_apple_4_18"], 本地加载
图像映射	third_person→base, wrist→wrist
状态 / 动作	state=observation.state, actions=action
语言提示	prompt=prompt, 并开启 prompt_from_task=True
数据读取方式	local_files_only=True, 直接消费本地 LeRobot 数据集
初始化权重	CheckpointWeightLoader
归一化资产	复用已有 assets 中的归一化统计量
批大小	batch_size=4
数据线程数	num_workers=2
训练步数	num_train_steps=50000
保存间隔	save_interval=10000
运行策略	overwrite=False, resume=True, wandb_enabled=False

其中有三项设置需要重点提及：

(1) **动作块长度设为 32**。与原论文动作视界更长的设定相比，本文将 `action_horizon` 设为 32。对于桌面抓取这种局部、快速闭环任务，更短的动作块有利于减少单次预测覆盖的开环时间，降低接触误差积累；同时在后续部署中，也更容易结合“执行一部分动作后重新观测再推理”的策略。

(2) **最大 token 长度设为 200**。虽然本文任务文本提示较短，但由于 $\pi_{0.5}$ 会把状态与多模态条件统一编码到序列中，保留较大的 token 容量可避免在迁移过程中因输入裁剪而损失条件信息。这一点也与本地实现中 `pi05=True` 时默认提升 `max_token_len` 的逻辑一致。

(3) **加载基础权重而非从零训练**。训练配置通过 `weight_loader` 加载已有的 `pi05_base` 参数，再结合本文任务数据进行继续训练。对于仅有 30 条示教数据的小规模数据集而言，这是获得可用性能的关键前提：视觉主干与语言主干提供通用感知能力，动作专家与相关层在任务数据上完成针对性调整。

4.2.3 训练产物与 checkpoint 组织

本文训练得到的模型保存在 `checkpoint` 目录下。从 `checkpoint` 元信息可以看到，保存内容主要分为三部分：

1. **assets**：保存与训练数据相关的归一化统计量；
2. **params**：保存可直接用于推理的模型参数；
3. **train_state**：保存训练态所需的优化器与其他状态（在推理部署时并非必须）。

这种组织方式与 `checkpoint` 管理脚本 `checkpoints.py` 中的实现完全一致。训练脚本在保存时会主动把适合推理的参数从完整训练状态中拆分出来，这样部署侧只需加载 `params` 与必要的归一化资产即可，不必携带优化器等冗余内容。最终目录中出现 49999 而非 50000，说明训练步数采用从零开始计数的方式，最后一次保存对应的是第 49999 步。

4.2.4 本文对 $\pi_{0.5}$ 的针对性修改

结合本地代码与实验流程，本文对 $\pi_{0.5}$ 的工作并不只是“换数据训练”，而是包含了以下几类面向 Franka 桌面场景的针对性适配：

1. **平台适配**：将 Franka 单臂 + 二指夹爪的八维状态/动作，与 LeRobot 标准数据集字段对齐，再进一步映射到 OpenPI 的 `images/state/actions/prompt` 输入接口；
2. **观测适配**：从原始移动机器人多相机设定，收缩到“第三人称 + 腕部相机”的双视角组合，保留对目标定位和精细接触都最关键的视觉信息；
3. **部署适配**：不直接照搬整段动作块开环执行，而是针对真实接触任务设计了更频繁的“观测–推理–执行”闭环策略；
4. **实验分析适配**：将原论文面向开放家庭场景的泛化指标，转换为本文任务更关心的抓取成功率、掉落/滑落现象以及失败后的再搜索能力。

因此，本文的算法实验并不是对已有开源代码的被动使用，而是以毕业设计场景为目标，对通用 VLA 管线做了一次具有工程与方法双重意义的落地化重构。

4.3 在线推理执行策略与实机效果分析

4.3.1 两种在线执行脚本的差异

训练完成后，本文分别使用远端推理脚本 `openpi_remote_client.py` 与分段闭环脚本 `openpi_remote_client_chunked.py` 进行实机部署。两者都通过 WebSocket 与 OpenPI 远端推理服务通信，并向 Franka 发送关节与夹爪指令，但其控制闭环方式明显不同。

前者采用**推理线程与控制线程解耦**的方式：推理线程不断发送最新观测并获取新的动作块，控制线程则以固定频率执行“当前最新的有效动作”。如果某些动作在推理返回时已经过期，脚本会直接将其丢弃。这样的设计在理论上可以提高系统利用率，但也意味着机器人在执行阶段可能使用的是一段**并非基于最新视觉反馈重新规划得到的动作序列**。

后者则采用**同步分段闭环**的方式：每次先读取一次机器人观测，推理得到一整个动作块；然后只执行该动作块中的第 3 到第 10 个动作（脚本中为 1-based 索引的动作 3–10）；执行完成后立刻重新读取新观测并再次推理。若系统控制频率取 30 Hz，则一次执行窗口大约只覆盖

$$\Delta t \approx \frac{8}{30} \text{ s} \approx 0.27 \text{ s}, \quad (4-5)$$

这使得策略几乎以“短时开环 + 高频重规划”的方式运行。虽然单次推理次数更多，但它显著增强了接触任务中的在线修正能力。

表 4-2 两种在线执行脚本的部署方式对比

脚本	控制方式	典型行为特征	实验现象
<code>openpi_remote_client.py</code>	异步推理 + 固定频率执行最新动作块	动作可能在返回时部分过期，执行更偏开环	成功率很低，约 5%，苹果常滑走
<code>openpi_remote_client_chunked.py</code>	同步分段推理，每次只执行动作块的一小段	高频“观测-执行-再观测-再执行”闭环	成功率接近 100%，抓取稳定显著提升

4.3.2 成功率差异的原因分析

本文实验中最突出的现象是：采用 `openpi_remote_client.py` 时，抓取成功率仅约为 5%；而改用 `openpi_remote_client_chunked.py` 后，成功率大幅提升，接近 100%。二者使用的是同一个训练后的模型，因此这一差异主要不是由模型参数本身导致，而是由**在线闭环方式**决定的。

结合任务过程，可以将原因概括为以下三点。

(1) 桌面抓取是强接触敏感任务，不能长时间开环。苹果表面光滑、受力后易滚动或侧滑。机械臂在接近苹果、闭合夹爪以及抬升初期这几个阶段，对末端相对位姿误差极为敏感。若模型依据稍早的观测生成一整段动作，而环境在执行中已经发生细小变化（例如苹果被碰动、夹爪接触点偏移），则后续尚未执行完的动作就会迅速失效。异步脚本虽然也会丢弃明显过期动作，但本质上仍倾向于执行一段较长的开环计划因此容易出现“看起来已经接近苹果，但夹取瞬间发生滑移”的现象。

(2) 分段执行将动作块预测转化为 receding horizon 控制。`openpi_remote_client_chunked.py` 并不一次吃完整个动作块，而是只截取其中一小段执行，然后立即重新获取观测并调用策略。这相当于把动作块预测变成了**滚动时域控制**：模型始终在最近一次观测的条件下生成接下来的局部动作，因此能够及时修正末端位置、夹爪开合时机以及抬升方向。对于抓取任务来说，这比单纯追求动作连续更重要。

(3) 闭环重推理显著提升了失败恢复能力。桌面抓取中的错误并不总是“彻底失败”，很多时候只是局部偏差，例如夹爪没有完全包住苹果、闭合时苹果稍微滑出、抬升初期苹果从夹爪边缘滚落等。同步分段推理使模型能在极短时间内再次观察当

前状态，并重新决定是继续抬升、重新对准，还是回退再抓。因此，它不仅提高了首次抓取的成功率，也增强了局部失败后的再恢复能力。

4.3.3 实验现象与任务完成质量

从实验现象看，两种执行脚本在接近目标与搬运路径平滑性方面都表现出较好的运动质量，说明训练得到的策略确实学习到了较稳定的桌面抓取动作模式；差异主要集中在抓取接触瞬间及其后的短时稳定保持阶段。采用异步脚本时，机械臂通常能够移动到苹果附近，但夹爪闭合与抬升的配合不够及时，苹果常在指尖边缘滑走，导致任务在最后一步失败。采用分段闭环脚本后，模型可以在每小段执行后重新观察苹果与夹爪的相对位置，因此显著降低了这种接触阶段的误差放大。

这一结果说明，对使用动作块输出的 VLA 而言，部署策略本身与模型结构同样重要。即使训练得到的动作块质量较高，若部署阶段仍以偏开环的方式执行较长动作序列，实机接触任务也可能出现明显性能下降；而将动作块推理嵌入更紧的闭环中，往往能够释放模型已有的感知-控制能力。这一结论对于今后将 VLA 部署到高精度桌面装配、插接、对准等任务中也具有普遍意义。

4.4 模型变体设计、效果预估与行为观察

在完成基础版 $\pi_{0.5}$ Franka 苹果抓取实验后，本文进一步从模型训练成本、泛化能力与失败恢复行为的角度，考虑若干可行的模型变体。需要说明的是，本节内容同时包含本文已有实验观察与基于结构机理给出的效果预估两类结论，二者在表述上将尽量区分。

4.4.1 全量微调基线

本文当前最主要的工作模型，是基于 $\pi_{0.5}$ 基础权重进行的任务微调版本。该版本能够充分适配桌面苹果抓取任务，在采用分段闭环部署时表现稳定，可作为全文实验的主基线模型。其优点是表达能力完整、上下文利用充分，缺点则是训练代价相对较高，对显存和训练稳定性要求更高。

4.4.2 冻结部分主干的轻量化变体

结合 `pi0_config.py` 中提供的 `get_freeze_filter()` 机制，可以自然构造“冻结部分主干，仅训练动作专家或少量适配参数”的变体。这类变体在小规模任务数

据上通常具有两个潜在优势：一是训练更稳定，二是参数更新范围更小，不容易破坏原有的视觉语义先验。

对本文任务而言，可考虑以下两种冻结策略：

1. **冻结视觉语言主干，仅微调动作专家与输出层。**该策略适合当目标任务与预训练语义差距不大、而主要差别在于机器人动力学与动作分布时。预期表现为：收敛更快、显存占用更小，但若桌面场景中的相机视角、苹果外观或背景干扰与基础模型经验差别较大，则最终上限可能低于全量微调。
2. **冻结视觉编码器，仅微调语言相关层和动作层。**该策略保留通用视觉特征，允许语言条件与控制头对具体任务进一步适配。预期上，它对“固定物体类别、固定相机视角”的桌面抓取任务可能较为合适，因为图像分布变化有限，而动作与任务条件需要更强适配。

从工程角度看，冻结部分主干尤其适合本文这种**样本规模小、任务较窄、部署目标明确**的毕业设计场景。即使其最终成功率略低于全量微调，若能够明显降低训练资源需求，也具备较高实用价值。因此，本节将其视作后续非常值得系统验证的一类轻量化路线。

4.4.3 short_memory 版本的行为特征

除基础版模型外，本文还观察到 short_memory 版本的 $\pi_{0.5}$ 在桌面抓取中同样表现良好，并呈现出一个非常有代表性的行为现象：**当模型发现自己没有成功夹住苹果时，会迅速返回去重新寻找苹果并再次尝试抓取。**这一现象具有重要的研究意义。

从机制上分析，short_memory 版本可以理解为：模型在决策时更依赖最近时刻的观测，而较少依赖较长时间窗口内的历史动作或历史状态。对于长时程任务，这种设计可能削弱全局计划能力；但对本文的桌面抓取任务而言，它反而可能带来以下好处：

1. **减小错误动作惯性。**当上一轮动作已经导致夹取失败时，较短记忆不会让策略继续沿着错误的内在轨迹“执拗执行”；
2. **增强近期视觉反馈权重。**模型会更快根据最新图像发现“苹果仍在桌面上、夹爪中为空”的事实；

3. 自然形成失败恢复。一旦识别到当前状态不符合“已抓取成功”的模式，模型更容易重新生成“回去找目标-再对准-再夹取”的动作序列。

换言之，`short_memory` 版本虽然不是传统意义上的显式恢复策略，但它在行为上表现出了一种基于近期观测的自发失败恢复能力。在论文撰写中，这一现象非常值得记录，因为它说明：对于接触敏感、短时闭环、高频反馈主导的桌面抓取任务，更长的时间记忆并不总是更优；适当缩短记忆窗口，反而可能减少错误累计，提高在线修正效率。

4.4.4 模型变体的综合比较

表 4-3 对本文已使用或建议进一步验证的几类模型变体做了归纳。需要再次强调，其中“已观察”表示已有实验现象支撑，而“预估”表示基于结构与任务机理的合理推断，后续仍需更系统的定量实验验证。

表 4-3 本文 $\pi_{0.5}$ 桌面抓取模型变体的效果分析

模型变体	结论性质	主要特点	预期/观察效果
基础版 $\pi_{0.5}$ 微调模型	已观察	能力完整，适合作为主基线	在分段闭环执行下抓取效果稳定，整体表现较好
冻结视觉语言主干，仅调动作专家	预估	训练代价低，保留原始语义先验	收敛更稳，但上限可能略低，尤其在视角偏移较大时
冻结视觉编码器，微调其余模块	预估	适合固定视角、固定对象类别场景	可能在小数据条件下取得较好的精度/成本折中
<code>short_memory</code> $\pi_{0.5}$	已观察 + 机理分析	更强调近期观测，弱化长时历史依赖	抓取失败后会迅速回去寻找苹果，表现出较强的再恢复能力

4.4.5 对后续工作的启示

从本文实验与上述变体分析可以得到两点启示。

第一，桌面抓取任务更依赖短周期闭环修正，而不是长时间开环规划。因此，无论是部署时采用分段执行，还是训练时引入更适合失败恢复的记忆结构，本质上都是在增强“看见当前状态后迅速修正动作”的能力。

第二，小样本专用任务中的最优模型，不一定是参数更新最多的模型。对于 30 条示教数据这样的小规模任务，冻结部分主干、仅微调动作专家或少量适配参数，很可能在训练资源与最终性能之间取得更好的平衡。未来如果继续扩展数据量、增加苹果种类、改变桌面背景或加入更多失败恢复示教，再对这些变体做定量对比，将有助于形成更完整的桌面 VLA 训练经验。

4.5 本章小结

本章围绕 $\pi_{0.5}$ 在 Franka 桌面苹果抓取任务上的落地过程，系统说明了模型原理、任务适配、训练设置、部署方式以及若干模型变体的行为特征。首先，从原理上分析了 $\pi_{0.5}$ 的分层推理框架、视觉语言主干与动作专家结构，以及离散动作预训练与连续动作后训练相结合的机制；随后结合本文的训练配置，说明了如何将 Franka 的双视角图像、八维状态/动作和语言提示映射到 OpenPI 训练接口，并基于 30 条示教数据完成微调训练；在实机部署中，实验进一步表明，同一模型在不同在线执行策略下会表现出显著差异，其中逐段闭环推理执行的 `chunked` 方案将抓取成功率从约 5% 提升到接近 100%，直接验证了“逐帧/逐段推理-执行-再推理-再执行”的有效性；最后，通过对冻结主干与 `short_memory` 变体的分析，本章指出桌面抓取任务尤其受益于高频反馈与失败恢复能力，这也为后续进一步改进桌面场景下的 VLA 训练与部署策略提供了方向。

结 论

本文结论……^[9]。

结论作为毕业设计（论文）正文的最后部分单独排写，但不加章号。结论是对整个论文主要结果的总结。在结论中应明确指出本研究的创新点，对其应用前景和社会、经济价值等加以预测和评价，并指出今后进一步在本研究方向进行研究工作的展望与设想。结论部分的撰写应简明扼要，突出创新性。阅后删除此段。

结论正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

参考文献

- [1] 余雄庆. 飞机总体多学科设计优化的现状与发展方向[J]. 南京航空航天大学学报, 2008(04): 417-426.
- [2] Hajela P, Bloebaum C L, Sobieszczanski-Sobieski J. Application of Global Sensitivity Equations in Multidisciplinary Aircraft Synthesis[J]. Journal of Aircraft, 1990, 27(12): 1002-110.
- [3] Black K, Brown N, Darpinian J, et al. pi0.5: A Vision-Language-Action Model with Open-World Generalization[J]. arXiv preprint arXiv:2504.16054, 2025.
- [4] Black K, Brown N, Driess D, et al. pi0: A Vision-Language-Action Flow Model for General Robot Control[J]. arXiv preprint arXiv:2410.24164, 2024.
- [5] Hugging Face. LeRobot[EB/OL]. 2025 [2026-05-05]. <https://github.com/huggingface/lerobot>.
- [6] Beyer L, Steiner A, Pinto A S, et al. PaliGemma: A Versatile 3B VLM for Transfer[J]. arXiv preprint arXiv:2407.07726, 2024.
- [7] Pertsch K, Stachowicz K, Ichter B, et al. FAST: Efficient Action Tokenization for Vision-Language-Action Models[J]. Robotics: Science and Systems, 2025.
- [8] Lipman Y, Chen R T Q, Ben-Hamu H, et al. Flow Matching for Generative Modeling[J]. arXiv preprint arXiv:2210.02747, 2022.
- [9] 张伯伟. 全唐五代诗格会考[M]. 南京: 江苏古籍出版社, 2002.

附 录

附录相关内容…

附录 A L^AT_EX 环境的安装

L^AT_EX 环境的安装。

附录 B B^IThesis 使用说明

B^IThesis 使用说明。

附录是毕业设计（论文）主体的补充项目，为了体现整篇文章的完整性，写入正文又可能有损于论文的条理性、逻辑性和精炼性，这些材料可以写入附录段，但对于每一篇文章并不是必须的。附录依次用大写正体英文字母 A、B、C……编序号，如附录 A、附录 B。阅后删除此段。

附录正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

致 谢

值此论文完成之际，首先向我的导师……

致谢正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。